Universidad de la Fuerzas Armadas - ESPE

Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Análisis Y Diseño de Software - NRC: 30723

Modelado de casos de uso de nivel 0 y apoyo al análisis con PlantUML

Grupo: 4

Tufiño Erick

Santi Jeancarlo

Cedeño Andrés

Profesor: Ing. Jenny Ruiz

Fecha: 04 de Mayo de 2026

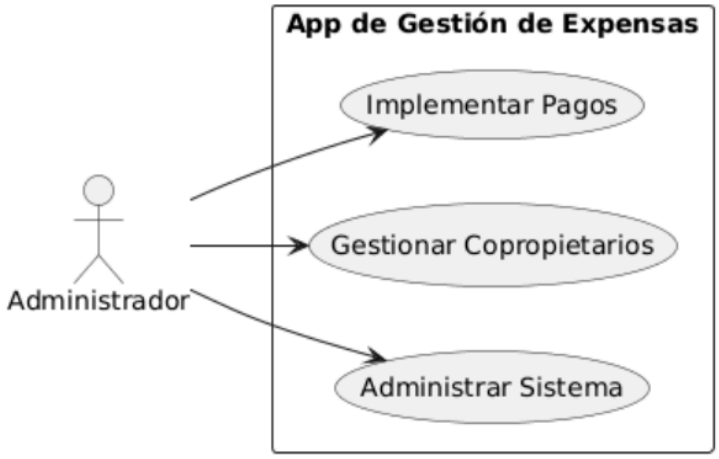
1. Requisitos Funcionales..... 3
2. Diagramas de Casos de Uso..... 4
3. Matriz de Trazabilidad.....7
4. Conclusión Teórica.....7

1. Requisitos Funcionales

Código	Requisito Funcional de nivel 0	Actor Principal	Caso de uso asociado
RF-01	El sistema permite gestionar el acceso, autenticación y configuración de perfiles de los usuarios. Los usuarios ingresan mediante credenciales de acceso, el sistema valida la información, crea una sesión activa y asigna permisos de acuerdo con el perfil correspondiente. Además, permite la recuperación de contraseña mediante un código de verificación enviado al correo registrado. El Administrador puede configurar los roles de los usuarios, asignando perfiles de Copropietario o Administrador, garantizando el control de acceso a las funcionalidades del sistema y la existencia de al menos un Administrador activo.	Administrador Copropietario	Administrar Sistema
RF-02	El sistema permite administrar la información de los copropietarios del conjunto residencial mediante operaciones de carga, consulta, modificación y eliminación de registros. Asimismo, posibilita la generación de reportes de consulta sobre información de pagos. El Copropietario solo puede acceder a sus propios datos, mientras que el Administrador puede consultar y generar reportes de todos los copropietarios registrados.	Administrador	Gestionar Copropietarios
RF-03	El sistema permite gestionar el proceso de registro, validación y control de pagos de expensas mensuales. El Copropietario registra el pago realizado mediante transferencia o depósito y sube el comprobante correspondiente. Posteriormente, el Administrador revisa y valida la información. Si existen deudas vencidas, el sistema detecta la morosidad y calcula automáticamente el recargo del 12 %. Una vez aprobado el pago, el sistema genera el comprobante digital, actualiza el estado de cuenta aplicando el pago a las deudas más antiguas y registra la transacción en el historial para auditoría y consultas futuras.	Copropietario Administrador	Implementar Pagos
RF-04	El sistema envía automáticamente notificaciones por WhatsApp a los copropietarios cuando ocurre un evento relacionado con pagos, como aprobación, rechazo o aplicación de sanción por mora. Para ello, valida que el copropietario tenga un número de teléfono registrado y con formato válido, genera el mensaje con la información correspondiente al evento y lo envía mediante el servicio configurado. Además, registra el resultado del envío para fines de auditoría. Si ocurre un fallo, el sistema realiza reintentos automáticos y, de persistir el error, deja constancia del fallo y notifica al Administrador.	Copropietario	Enviar Notificaciones

2. Diagramas de Casos de Uso

Caso de uso del Administrador



Código usado en PlantUML

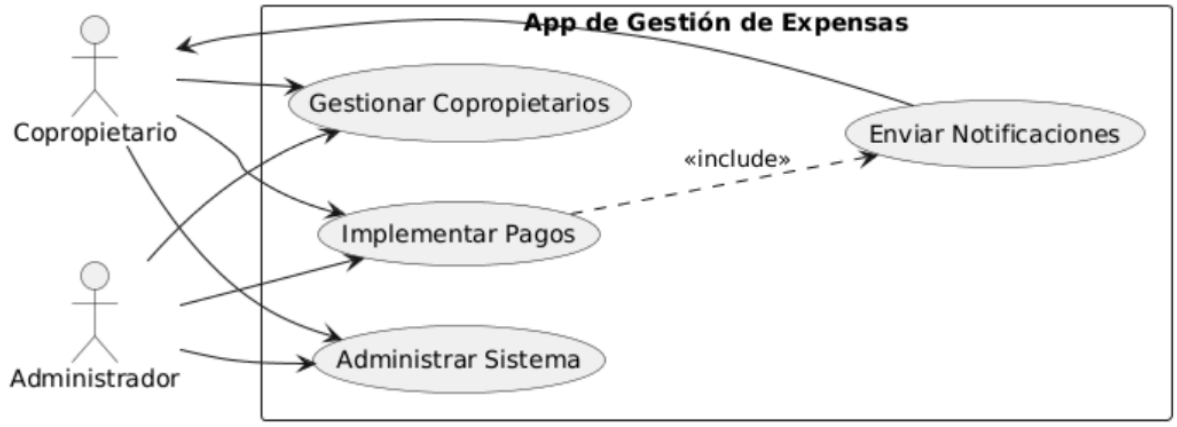
```
@startuml
left to right direction
actor Administrador
rectangle "App de Gestión de Expensas" {
 usecase "Administrar Sistema" as UC1
 usecase "Gestionar Copropietarios" as UC2
 usecase "Implementar Pagos" as UC3
}
Administrador --> UC1
Administrador  --> UC2
Administrador  --> UC3
@enduml
```

Caso de uso del Copropietario

Código usado en PlantUML

```
@startuml
left to right direction
actor Copropietario
rectangle "App de Gestión de Expensas" {
 usecase "Administrar Sistema" as UC1
 usecase "Gestionar Copropietarios" as UC2
 usecase "Implementar Pagos" as UC3
 usecase "Enviar Notificaciones" as UC4
}
Copropietario --> UC1
Copropietario --> UC2
Copropietario --> UC3
UC4 --> Copropietario
@enduml
```

Diagrama General de Casos de Uso del Nivel 0



Código usado en PlantUML

```
@startuml
left to right direction
actor Copropietario
actor Administrador

rectangle "App de Gestión de Expensas" {
 usecase "Administrar Sistema" as UC1
 usecase "Gestionar Copropietarios" as UC2
 usecase "Implementar Pagos" as UC3
 usecase "Enviar Notificaciones" as UC4
}
Copropietario --> UC1
Copropietario --> UC2
Copropietario --> UC3

Administrador --> UC1
Administrador --> UC2
Administrador --> UC3

UC4 --> Copropietario

UC3 ..> UC4 : <<include>>
@enduml
```

3. Matriz de Trazabilidad

RF	Caso de Uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF - 01	Administrar Sistema	Copropietario Administrador	Alta	El sistema autentica usuarios, recupera contraseñas, asigna permisos por rol y asegura la existencia de al menos un Administrador activo.	UC1
RF - 02	Gestionar Copropietarios	Copropietario Administrador	Alta	El sistema administra los datos de copropietarios y genera reportes, respetando las restricciones de acceso según el rol del usuario.	UC2
RF - 03	Implementar Pagos	Copropietario Administrador	Alta	El sistema registra, valida y procesa pagos de expensas, calcula mora, actualiza estados de cuenta y genera comprobantes digitales.	UC3
RF - 04	Enviar Notificaciones	Copropietario	Media	El sistema envía notificaciones automáticas por WhatsApp sobre eventos de pago y registra el resultado de cada envío.	UC4

4. Conclusión Teórica

La trazabilidad elaborada por mi grupo de trabajo entre los requisitos funcionales, actores, casos de uso y la evidencia de la herramienta PlantUML, nos permitió verificar que el modelo de nuestro nivel 0 mantiene coherencia con el alcance definido. Cada requisito funcional se encuentra representado por un caso de uso identificable, asociado a los actores que intervienen en el proceso y respaldado mediante su codificación en el diagrama UML. Esta relación favorece la consistencia del modelo, ya que reduce ambigüedades entre lo que el sistema debe hacer y su representación gráfica inicial. Además, la matriz de trazabilidad aporta utilidad al análisis porque facilita validar cobertura funcional, detectar omisiones tempranas y servir como base para las siguientes fases de diseño.